CO2-AT3_Questions-Slot-C

Register No: 192311369
Student: ADITHYA R
Assigned Question:

Q1. Debugging Selection Sort Code

The following Selection Sort code contains logical errors and produces incorrect output for some inputs. Carefully analyze the code, identify all errors in comparison and swapping logic, and explain their root causes. Apply systematic debugging to correct the algorithm step-by-step. Optimize the implementation by reducing unnecessary comparisons if possible. Analyze the corrected algorithm in terms of time complexity. Rewrite the final optimized code with proper structure and comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Solution :

1. Incorrect Comparison Operator

Given Code:

```
if arr[j] > arr[min_idx]:
```

Mistake:

- The condition uses ">" instead of "<", so the inner loop keeps track of the maximum element in the unsorted part, not the minimum.
- As a result, the algorithm repeatedly places the largest remaining element first, sorting the array in descending order instead of ascending order.

Correction:

```
if arr[j] < arr[min_idx]:
```

2. Root Cause of Incorrect Sorting

- Selection Sort is designed to select the minimum element from the unsorted portion and move it to the front.
- Using ">" reverses this intent, causing min_idx to always track the largest value instead of the smallest.
- Every swap therefore places the wrong element at position i, producing an incorrectly ordered (descending) array.

3. Unnecessary Swapping

Given Code:

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Mistake:

- This swap executes on every outer loop iteration, even when min_idx is already equal to i (the element is already in its correct position).
- This performs a redundant self-swap, adding unnecessary operations and reducing efficiency for nearly sorted arrays.

Correction:

```
if min_idx != i:
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

4. Reducing Unnecessary Comparisons

- The inner loop must still scan every element in the unsorted portion to correctly find the minimum, so the number of comparisons cannot be reduced below $O(n^2)$ in the general case.
- However, wrapping the swap in a min_idx != i check removes unnecessary write operations, which is a meaningful practical optimization even though it does not change the comparison count.

5. Readability Improvements

- Add a docstring and inline comments explaining each step.
- Use consistent indentation and spacing.
- Use descriptive variable names (min_idx, n) as already present, kept for clarity.
- Guard the swap step separately from the comparison step for clearer logic flow.

After correction:

```
def selection_sort(arr):
    """
    Sorts an array in ascending order using the
    Selection Sort algorithm.
    """
    n = len(arr)
    for i in range(n):
        min_idx = i  # Assume current index holds the minimum
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:  # Find the true minimum
                min_idx = j
        if min_idx != i:  # Swap only if a smaller element was found
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Why Incorrect Sorting Occurs

The algorithm correctly scans the unsorted portion of the array on each pass, but the comparison operator "arr[j] > arr[min_idx]" causes it to track the maximum element instead of the minimum. Consequently, min_idx always points to the largest remaining value, and each swap places that largest value at the front of the unsorted region. This inverts the intended ordering, so the array ends up sorted in descending order rather than ascending order. Additionally, performing the swap unconditionally (even when min_idx equals i) adds unnecessary operations without affecting correctness, but it is inefficient.

Time Complexity

- Best Case: $O(n^2)$ — comparisons are always made regardless of initial order; swaps are minimal (as low as 0 with the min_idx != i check).
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: O(1) — sorting is done in-place.